# Массивы в numpy

## Теория

```
(X - X.mean(0))
/ X.std(0)
```



numpy — основа почти всего в курсе: pandas, sklearn и PyTorch стоят на тех же массивах. Здесь — всё, что нужно к серии: форма и оси, маски, пропуски, распространение, сортировка, группы, окна и как обойтись без циклов. Код идёт подряд, как ячейки ноутбука; под ним, с чертой слева, — ровно то, что он печатает. За подробностями — документация numpy, раздел «NumPy: the absolute basics for beginners».

## Массив

Массив numpy — таблица чисел одного типа любой размерности. У него есть форма — сколько элементов по каждой оси, — и тип элементов.

```
import numpy as np
a = np.array([[5, 4, 3], [4, 4, 5]])
print(a)
print(a.shape, a.ndim, a.dtype)
print(np.arange(6).reshape(2, 3))
print(np.zeros(3), np.full(2, 7.5))
```

```
[[5 4 3]
 [4 4 5]]
(2, 3) 2 int64
[[0 1 2]
 [3 4 5]]
[0. 0. 0.] [7.5 7.5]
```

`shape` — форма, `ndim` — число осей, `dtype` — тип. `reshape` меняет форму, не трогая порядок чисел; одно измерение можно отдать на вычисление: `reshape(-1, 3)`. Ещё `np.linspace(0, 1, 5)` — пять равных шагов от 0 до 1 включительно.

## Индексы и срезы

Номера с нуля, по оси через запятую. Срез `a:b` берёт номера от `a` до `b`, не включая `b`; отрицательный номер считается с конца.

```
t = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 10, 11, 12]])
print(t[1, 2], t[-1, -1])
print(t[:, 1])
print(t[1:, ::2])
print(t[[0, 2]])
```

```
7 12
[ 2  6 10]
[[ 5  7]
 [ 9 11]]
[[ 1  2  3  4]
 [ 9 10 11 12]]
```

`t[:, 1]` — весь второй столбец, `::2` — через один, `t[[0, 2]]` — строки по списку номеров.

**Срез — не копия.** Он смотрит в те же данные, и запись в него меняет исходный массив:

```
row = t[0]
row[0] = 100
print(t[0])
safe = t[1].copy()
safe[0] = -1
print(t[1])
```

```
[100   2   3   4]
[5 6 7 8]
```

Функция, которой передали массив, не должна его портить: всё, что меняется на месте — `a.sort()`, `a[...] = ...`, `a += 1`, — делают на копии `a.copy()` или пишут без изменения на месте: `np.sort(a)`, `a + 1`.

## Маски

Сравнение массива даёт массив из `True` и `False` — маску. Ей выбирают элементы, по ней считают:

```
x = np.array([3, -1, 4, -1, 5, -9, 2])
m = x > 0
print(m)
print(x[m], m.sum(), m.mean())
print(x[(x > 0) & (x < 5)])
print(np.where(x > 0, x, 0))
```

```
[ True False  True False  True False  True]
[3 4 5 2] 4 0.5714285714285714
[3 4 2]
[3 0 4 0 5 0 2]
```

`m.sum()` — сколько `True`, `m.mean()` — их доля. Условия соединяют `&`, `|`, `~`, каждое в скобках.
`np.where(условие, a, b)` — поэлементно «если, то, иначе».
Есть ли хоть один и все ли — `m.any()`, `m.all()`.

## Оси

Счёт вдоль оси сворачивает её: у массива формы $(2, 3)$ сумма по оси 0 даёт три числа — по столбцам, по оси 1 — два, по строкам.

```
o = np.array([[5, 4, 3],
              [4, 4, 5]])
print(o.sum(axis=0), o.sum(axis=1), o.sum())
print(o.mean(axis=1), o.max(axis=0))
print(o.argmax(axis=1))
```

```
[9 8 8] [12 13] 25
[4.         4.33333333] [5 4 5]
[0 2]
```

**Правило: `axis` называет ту ось, которой не станет.** Для трёх осей — скажем, ученик, предмет, четверть — среднее по ученикам и четвертям даёт по числу на предмет:

```
g = np.arange(24).reshape(2, 3, 4)
print(g.mean(axis=(0, 2)))
print(g.sum(axis=2, keepdims=True).shape)
print((g % 5 == 0).any(axis=2))
```

```
[ 7.5 11.5 15.5]
(2, 3, 1)
[[ True  True  True]
 [ True False  True]]
```

`keepdims=True` оставляет свёрнутую ось длиной 1 — так результат удобно вычитать из исходного массива. `argmax` — номер наибольшего вдоль оси.

## Пропуски

Пропуск — `np.nan`, особое дробное число. Всё, что с ним считается, тоже становится `nan`, а сравнение с ним всегда ложно, даже `nan == nan`. Поэтому у счёта есть варианты с `nan`:

```
y = np.array([2.0, np.nan, 4.0, np.nan, 9.0])
print(y.mean(), np.nanmean(y), np.nanmax(y))
print(np.isnan(y), np.isnan(y).sum())
print(y[~np.isnan(y)])
```

```
nan 5.0 9.0
[False  True False  True False] 2
[2. 4. 9.]
```

`np.isnan` — маска пропусков; `nanmean`, `nansum`, `nanmax`, `nanmin`, `nanmedian` их пропускают. Если вдоль оси одни пропуски, `nanmean` даст `nan` и предупреждение.

## Типы

Тип выбирается по входу и держится при счёте. Две ловушки. Целые маленькие типы переполняются молча: у `uint8`, типа картинок, числа от 0 до 255, и разность уходит по кругу. И деление целых `/` даёт дробь, а `//` — целую часть.

```
p = np.array([10, 200], dtype=np.uint8)
print(p[0] - p[1], np.diff(p))
q = p.astype(np.int64)
print(q[0] - q[1], np.diff(q))
print(np.array([7, 8]) / 2, np.array([7, 8]) // 2)
```

```
66 [190]
-190 [190]
[3.5 4. ] [3 4]
```

`astype` переводит в другой тип. Дроби сравнивают с допуском: `0.1 + 0.2 == 0.3` ложно, а `np.isclose(0.1 + 0.2, 0.3)` — истинно.

## Распространение

Арифметика идёт поэлементно, а массивы разной формы растягиваются друг под друга — по-английски это broadcasting. Правило: формы сравнивают с конца, и по каждой оси длины должны совпадать или одна из них равна 1 — она и растянется.

```
col = np.array([[0], [10], [20]])
row = np.array([1, 2, 3])
print(col.shape, row.shape)
print(col + row)
X = np.array([[1.0, 4.0], [3.0, 8.0], [5.0, 6.0]])
print(X - X.mean(axis=0))
```

```
(3, 1) (3,)
[[ 1  2  3]
 [11 12 13]
 [21 22 23]]
[[-2. -2.]
 [ 0.  2.]
 [ 2.  0.]]
```

Столбец $(3, 1)$ и строка $(3, )$ дали таблицу $(3, 3)$ всех сумм. Вычесть из каждого столбца его среднее — одна строка: форма `X.mean(axis=0)` равна $(2, )$ и растягивается на все строки.

**Все пары строк сразу.** Новая ось `None` вставляет измерение длины 1:

```
P = np.array([[0, 0], [3, 4]])
Q = np.array([[0, 0], [1, 1], [6, 8]])
d = P[:, None, :] - Q[None, :, :]
print(d.shape)
print(np.sqrt((d ** 2).sum(axis=2)))
```

```
(2, 3, 2)
[[ 0.          1.41421356 10.        ]
 [ 5.          3.60555128  5.        ]]
```

Форма $(2, 1, 2)$ против $(1, 3, 2)$ даёт $(2, 3, 2)$: каждая точка `P` против каждой точки `Q`, по каждой координате. Сумма по последней оси — квадраты расстояний. Памяти уходит $n \cdot m \cdot d$ чисел: при миллионах это ещё нормально.

## Сортировка

```
s = np.array([30, 10, 20, 10])
print(np.sort(s), np.argsort(s, kind="stable"))
print(np.argsort(-s, kind="stable"))
print(s[np.argsort(-s, kind="stable")[:2]])
```

```
[10 10 20 30] [1 3 2 0]
[0 2 1 3]
[30 20]
```

`argsort` — номера элементов в порядке возрастания; по убыванию — `argsort(-s)`. При равных значениях порядок задаёт `kind="stable"`: равные остаются в исходном порядке, то есть меньший номер раньше. Без него порядок равных не обещан. Сортировка строк по столбцу — `X[np.argsort(X[:, 1])]`.

## Группы и счёт

Группа задана номером от 0. `bincount` считает, сколько раз встретился каждый номер, а с `weights` — сумму весов по номерам:

```
grp = np.array([0, 2, 0, 1, 2, 2])
val = np.array([4.0, 1.0, 6.0, 3.0, 2.0, 3.0])
print(np.bincount(grp))
s = np.bincount(grp, weights=val)
print(s)
print(s / np.bincount(grp))
print(np.bincount(grp, minlength=5))
```

```
[2 1 3]
[10.  3.  6.]
[5. 3. 2.]
[2 1 3 0 0]
```

Суммы делить на количества — средние по группам. `minlength` добавляет группы, которых в данных нет: у них счётчик 0, и деление даст `nan` с предупреждением — такое деление делают `np.divide(s, c, out=..., where=c > 0)`. Если группы — не номера, а любые значения, номера даёт `unique`:

```
w = np.array(["b", "a", "b", "c", "a", "b"])
u, inv, cnt = np.unique(w, return_inverse=True,
                        return_counts=True)
print(u, inv, cnt)
```

```
['a' 'b' 'c'] [1 0 1 2 0 1] [2 3 1]
```

`u` — различные значения по возрастанию, `inv` — номер значения у каждого элемента, `cnt` — сколько раз каждое встречается. Два номера в один — `i * k + j`: так `bincount` считает пары, а `reshape(k, k)` раскладывает результат таблицей.

## Накопления и окна

```
v = np.array([3, 1, 4, 1, 5, 9, 2])
c = np.cumsum(v)
print(c, np.diff(v))
print(c[3:] - np.r_[0, c[:-4]])
swv = np.lib.stride_tricks.sliding_window_view
win = swv(v, 4)
print(win.shape)
print(win.max(axis=1), win.mean(axis=1))
```

```
[ 3  4  8  9 14 23 25] [-2  3 -3  4  4 -7]
[ 9 11 19 17]
(4, 4)
[4 5 9 9] [2.25 2.75 4.75 4.25]
```

`cumsum` — накопленные суммы, `diff` — разности соседних. Сумма окна длины 4 — разность накопленных сумм на расстоянии 4. `sliding_window_view(v, 4)` показывает все окна строками, не копируя данных; дальше — любой счёт по оси 1. Серии одинаковых значений находят по `diff`: `np.diff(np.r_[0, m, 0])` для маски `m` из 0 и 1 даёт 1 в начале каждой серии и $-1$ после её конца.

## Линейная алгебра

`@` — произведение матриц и матрицы на вектор, `np.linalg.norm` — длина:

```
A = np.array([[1, 2], [3, 4]])
print(A @ np.array([1, -1]))
V = np.array([[3.0, 4.0], [1.0, 0.0], [0.0, 2.0]])
n = np.linalg.norm(V, axis=1)
print(n)
U = V / n[:, None]
print((U @ U.T).round(2))
```

```
[-1 -1]
[5. 1. 2.]
[[1.  0.6 0.8]
 [0.6 1.  0. ]
 [0.8 0.  1. ]]
```

Строки, делённые на свои длины, дают единичные векторы, и `U @ U.T` — таблица косинусов всех пар. Решить систему $Ax = b$ — `np.linalg.solve(A, b)`. Пары $i < j$ без повторов — `np.triu_indices(n, 1)`.

## Без циклов и файлы

Цикл по элементам в питоне в десятки раз медленнее одной операции над массивом: операция идёт в скомпилированном коде. На миллионе чисел это разница между долей секунды и минутой. Поэтому сначала ищут, какой операцией над целыми массивами задача решается: маска, ось, распространение, `bincount`, `cumsum`, сортировка. `np.vectorize` и `apply_along_axis` цикл не убирают — они прячут его внутри.

Массивы хранят в файлах `.npy`, а несколько сразу — в `.npz`:

```
np.savez("demo.npz", a=a, y=y)
d = np.load("demo.npz")
print(sorted(d.files), d["a"].shape)
rng = np.random.default_rng(42)
print(rng.integers(0, 10, 5), rng.permutation(4))
```

```
['a', 'y'] (2, 3)
[0 7 6 4 4] [3 0 1 2]
```

`np.load` у `.npz` даёт словарь массивов по именам. Случайные числа — через генератор с зерном `default_rng(42)`: с тем же зерном выходят те же числа.